

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



ARBI RAMADHAN

109082500114

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1. Selection Sort

Selection Sort adalah algoritma pengurutan yang bekerja dengan cara memilih elemen terkecil (atau terbesar) dari bagian array yang belum terurut, kemudian menukarnya dengan elemen di posisi paling awal dari bagian tersebut. Proses ini diulang secara bertahap hingga seluruh array terurut.

2. Insertion Sort

Insertion sort adalah algoritma pengurutan yang bekerja dengan cara mengambil satu elemen dari bagian array yang belum terurut, kemudian menyisipkannya ke posisi yang tepat di bagian array yang sudah terurut. Algoritma ini mirip dengan cara orang mengurutkan kartu remi di tangan.

B. Guided

1. Program Selection Sort pada Array

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int

    for i = 0; i < len(arr) - 1; i++ { //perulangan
    luar
        idx_min = i;
        for j = i + 1; j < len(arr); j++ { //perulangan
        dalam
            if arr[j] < arr[idx_min] { //kondisional
                idx_min = j
            }
        }
        if idx_min != i { //swap
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-%d:
", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    SelectionSortArray(&arrAngka)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}
```

Output :

```
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan\Week 12 - Modul 14\Guided\Guided-1\guided1.go"
Masukkan data angka indeks ke-0: 3
Masukkan data angka indeks ke-1: 7
Masukkan data angka indeks ke-2: 4
Masukkan data angka indeks ke-3: 1
Masukkan data angka indeks ke-4: 8
Masukkan data angka indeks ke-5: 9
Masukkan data angka indeks ke-6: 4
Masukkan data angka indeks ke-7: 2

=== SEBELUM SORTING ===
3 7 4 1 8 9 4 2
=== SETELAH SORTING ===
1 2 3 4 4 7 8 9
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan>
```

2. Program Selection Sort pada Struct

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK      float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ {
            if sMHS[j].IPK < sMHS[idx_min].IPK {
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa
```

```

        for i := 0; i < len(structMHS); i++ {
            fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n", i)
            fmt.Print("Nama : ")
            fmt.Scan(&structMHS[i].nama)
            fmt.Print("NIM : ")
            fmt.Scan(&structMHS[i].nim)
            fmt.Print("IPK : ")
            fmt.Scan(&structMHS[i].IPK)
        }
        fmt.Println()

        fmt.Println(" === SEBELUM SORTING === ")
        for i := 0; i < len(structMHS); i++ {
            fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
            fmt.Printf("Nama : %s\n", structMHS[i].nama)
            fmt.Printf("NIM : %s\n", structMHS[i].nim)
            fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
        }
        fmt.Println("=====")
        fmt.Println()

        SelectionSortArray(&structMHS)
        fmt.Println(" === SETELAH SORTING === ")
        for i := 0; i < len(structMHS); i++ {
            fmt.Printf("Nama : %s\n", structMHS[i].nama)
            fmt.Printf("NIM : %s\n", structMHS[i].nim)
            fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
        }
        fmt.Println("=====")
        fmt.Println()
    }
}

```

Output :

```

PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan\Week 12 - Modul 14\Guided\Guided-2\guided2.go"
--- Masukkan Data Mahasiswa ke-0 ---
Nama : dhimas
NIM : 123
IPK : 3.21
--- Masukkan Data Mahasiswa ke-1 ---
Nama : hafizh
NIM : 234
IPK : 2.46
--- Masukkan Data Mahasiswa ke-2 ---
Nama : fathur
NIM : 345
IPK : 3.58
--- Masukkan Data Mahasiswa ke-3 ---
Nama : rahman
NIM : 456
IPK : 1.57
--- Masukkan Data Mahasiswa ke-4 ---
Nama : masdim
NIM : 567
IPK : 3.96

```

```
=== SEBELUM SORTING ===  
--- Data Mahasiswa Ke-0 ---  
Nama : dhimas  
NIM : 123  
IPK : 3.21  
--- Data Mahasiswa Ke-1 ---  
Nama : hafizh  
NIM : 234  
IPK : 2.46  
--- Data Mahasiswa Ke-2 ---  
Nama : fathur  
NIM : 345  
IPK : 3.58  
--- Data Mahasiswa Ke-3 ---  
Nama : rahman  
NIM : 456  
IPK : 1.57  
--- Data Mahasiswa Ke-4 ---  
Nama : masdim  
NIM : 567  
IPK : 3.96  
=====
```

```
=== SETELAH SORTING ===  
Nama : rahman  
NIM : 456  
IPK : 1.57  
Nama : hafizh  
NIM : 234  
IPK : 2.46  
Nama : dhimas  
NIM : 123  
IPK : 3.21  
Nama : fathur  
NIM : 345  
IPK : 3.58  
Nama : masdim  
NIM : 567  
IPK : 3.96  
=====
```

```
❖ PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan>
```

3. Program Inselection Sort pada Array Descending

```
package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n - 1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j - 1] {
            T[j] = T[j - 1]
            j--
        }
        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data: ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println("\nData sebelum sorting: ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nData setelah sorting (Descending):")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}
```

Output :

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell + v [ ] [ ] ... | [ ] [ ] X

PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114
_Arbi-Ramadhan\Week 12 - Modul 14\Guided\Guided-3\guided3.go"
Masukkan jumlah data: 5
Masukkan data:
7 3 9 1 5

Data sebelum sorting:
7 3 9 1 5

Data setelah sorting (Descending):
9 7 5 3 1
❖ PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan>

```

4. Program Insertion Sort Pada Struct Mahasiswa

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    ipk  float64
}

type arrMhs [100]mahasiswa

func insertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama < temp.nama {
            T[j + 1] = T[j]
            j--
        }
        T[j + 1] = temp
    }
}

func main() {
    var data arrMhs
```



```

var n, i int

fmt.Print("Masukkan jumlah mahasiswa: ")
fmt.Scan(&n)

for i = 0; i < n; i++ {
    fmt.Println("\nData mahasiswa ke-", i + 1)

    fmt.Print("Nama: ")
    fmt.Scan(&data[i].nama)

    fmt.Print("NIM: ")
    fmt.Scan(&data[i].nim)

    fmt.Print("IPK: ")
    fmt.Scan(&data[i].ipk)
}

fmt.Println("\nData sebelum sorting: ")
for i = 0; i < n; i++ {
    fmt.Println(data[i].nama, data[i].nim,
data[i].ipk)
}

insertionSortStruct(&data, n)

fmt.Println("\nData setelah sorting (Descending
berdasarkan Nama): ")
for i = 0; i < n; i++ {
    fmt.Println(data[i].nama, data[i].nim,
data[i].ipk)
}
}

```

Output :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] ... [ ] [ ] X

PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114
_Arbi-Ramadhan\Week 12 - Modul 14\Guided\Guided-4\guided4.go"
Masukkan jumlah mahasiswa: 3

Data mahasiswa ke- 1
Nama: rere
NIM: 101
IPK: 4

Data mahasiswa ke- 2
Nama: regita
NIM: 102
IPK: 4

Data mahasiswa ke- 3
Nama: reguta
NIM: 103
IPK: 4

Data sebelum sorting:
rere 101 4
regita 102 4
reguta 103 4

Data setelah sorting (Descending berdasarkan Nama):
rere 101 4
reguta 103 4
regita 102 4
❖ PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> |
```

C. Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func selectionSort(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        arr[i], arr[idxMin] = arr[idxMin], arr[i]
    }
}
```

```

func main() {
    var n int
    fmt.Scan(&n)

    for daerah := 1; daerah <= n; daerah++ {
        var m int
        fmt.Scan(&m)

        rumah := make([]int, m)
        for i := 0; i < m; i++ {
            fmt.Scan(&rumah[i])
        }

        selectionSort(rumah, m)

        for i := 0; i < m; i++ {
            fmt.Printf("%d ", rumah[i])
        }
        fmt.Println()
    }
}

```

Output :

```

PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114_
Arbi-Ramadhan\Week 12 - Modul 14\Unguided\Unguided-no1\unguided1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan>

```

Penjelasan :

Pada kode program ini saya membuat sebuah alat untuk menyusun nomor-nomor rumah kerabat di setiap daerah secara terurut membesar menggunakan algoritma selection sort. Program ini membaca jumlah daerah terlebih dahulu, kemudian untuk setiap daerah membaca jumlah rumah dan nomor-nomor rumahnya, lalu mengurutkannya sebelum ditampilkan. Pertama, program mendeklarasikan variabel `n` untuk menyimpan banyaknya daerah. Program membaca nilai `n` menggunakan `fmt.Scan(&n)`. Kemudian program melakukan perulangan sebanyak `n` kali untuk memproses setiap daerah .

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main

import "fmt"

func selectionSort(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        arr[i], arr[idxMin] = arr[idxMin], arr[i]
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for daerah := 1; daerah <= n; daerah++ {
        var m int
        fmt.Scan(&m)

        rumah := make([]int, m)
        for i := 0; i < m; i++ {
            fmt.Scan(&rumah[i])
        }

        selectionSort(rumah, m)

        for i := 0; i < m; i++ {
            if rumah[i]%2 == 1 {
                fmt.Printf("%d ", rumah[i])
            }
        }

        for i := m - 1; i >= 0; i-- {
            if rumah[i]%2 == 0 {
                fmt.Printf("%d ", rumah[i])
            }
        }
        fmt.Println()
    }
}
```

Output :



```
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan\Week 12 - Modul 14\Unguided\Unguided-no2\unguided2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan>
```

Penjelasan :

Pada kode program ini saya membuat sebuah alat untuk menyusun nomor rumah kerabat dengan aturan khusus: nomor ganjil ditampilkan terlebih dahulu secara terurut membesar, kemudian nomor genap ditampilkan secara terurut mengecil. Program ini mengikuti petunjuk yang diberikan, yaitu memisahkan bilangan ganjil dan genap ke dalam dua array yang berbeda saat pembacaan data. Program mendeklarasikan variabel `n` untuk menyimpan banyaknya daerah, kemudian membacanya menggunakan `fmt.Scan(&n)`. Program kemudian melakukan perulangan sebanyak `n` kali untuk memproses setiap daerah. Untuk setiap daerah, program membaca nilai `m` (banyaknya rumah kerabat). Program membuat dua slice kosong: ganjil dan genap menggunakan `var ganjil, genap []int`.

3. Soal Unguided 3

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 1 7 19 37 5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```
package main

import (
    "fmt"
)

func insertionSort(arr []int, n int) {
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}
```



```

func main() {
    var data []int
    var input int

    for {
        fmt.Scan(&input)
        if input < 0 {
            break
        }
        data = append(data, input)
    }

    n := len(data)

    insertionSort(data, n)

    for i := 0; i < n; i++ {
        fmt.Printf("%d ", data[i])
    }
    fmt.Println()

    if n <= 1 {
        fmt.Println("Data berjarak tidak tetap")
        return
    }

    jarak := data[1] - data[0]
    berjarakTetap := true

    for i := 2; i < n; i++ {
        if data[i]-data[i-1] != jarak {
            berjarakTetap = false
            break
        }
    }

    if berjarakTetap {
        fmt.Printf("Data berjarak %d\n", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output :



```
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan\Week 12 - Modul 14\Unguided\Unguided-no3\unguided3.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan> go run "c:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan\Week 12 - Modul 14\Unguided\Unguided-no3\unguided3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\Data kuliah\Semester 2\Alpro2\109082500114_Arbi-Ramadhan>
```

Penjelasan :

Pada kode program ini saya membuat sebuah alat untuk membaca data bilangan bulat, mengurutkannya menggunakan algoritma insertion sort, kemudian memeriksa apakah data yang telah terurut memiliki jarak (selisih) yang sama antar elemen yang berurutan. Program menggunakan perulangan for tanpa kondisi untuk membaca bilangan satu per satu menggunakan `fmt.Scan(&input)`. Setiap kali membaca bilangan, program memeriksa apakah bilangan tersebut negatif. Jika ya, perulangan dihentikan dengan `break`. Jika bilangan non-negatif (≥ 0), bilangan tersebut ditambahkan ke slice data menggunakan `append`. Proses ini berlanjut hingga menemukan bilangan negatif sebagai sentinel (penanda akhir).

4. Soal Unguided 4

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Lengkapisubprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

```

procedure DaftarkanBuku(in/out pustaka : DaftarBuku, n : integer)
{I.S. sejumlah n data buku telah siap para piranti masukan
 F.S. n berisi sebuah nilai, dan pustaka berisi sejumlah n data buku}

procedure CetakTerfavorit(in pustaka : DaftarBuku, in n : integer)
{I.S. array pustaka berisi n buah data buku dan belum terurut
 F.S. Tampilan data buku (judul, penulis, penerbit, tahun)
terfavorit, yaitu memiliki rating tertinggi}

procedure UrutBuku( in/out pustaka : DaftarBuku, n : integer )
{I.S. Array pustaka berisi n data buku
 F.S. Array pustaka terurut menurun/mengecil terhadap rating.
Catatan: Gunakan metoda Insertion sort}

procedure Cetak5Terbaru( in pustaka : DaftarBuku, n integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan 5 judul buku dengan rating tertinggi
 Catatan: Isi pustaka mungkin saja kurang dari 5}

procedure CariBuku(in pustaka : DaftarBuku, n : integer, r : integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan salah satu buku (judul, penulis, penerbit, tahun,
eksemplar, rating) dengan rating yang diberikan. Jika tidak ada buku
dengan rating yang ditanyakan, cukup tuliskan "Tidak ada buku dengan
rating seperti itu". Catatan: Gunakan pencarian biner/belah dua.}

```

Jawaban :

```

package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

var pustaka DaftarBuku
var nPustaka int

// DaftarkanBuku mengisi array pustaka dengan sejumlah
n data buku dari masukan
func DaftarkanBuku(pustaka *DaftarBuku, n *int) {

```

```

        fmt.Scan(n) // baca jumlah buku
        for i := 0; i < *n; i++ {
            fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
                &pustaka[i].penulis,
                &pustaka[i].penerbit,
                &pustaka[i].eksemplar, &pustaka[i].tahun,
                &pustaka[i].rating)
        }
    }

    // CetakTerfavorit menampilkan buku dengan rating tertinggi
    func CetakTerfavorit(pustaka DaftarBuku, n int) {
        if n == 0 {
            return
        }
        idxMax := 0
        for i := 1; i < n; i++ {
            if pustaka[i].rating > pustaka[idxMax].rating {
                idxMax = i
            }
        }
        fmt.Printf("%s %s %s %d\n", pustaka[idxMax].judul,
            pustaka[idxMax].penulis,
            pustaka[idxMax].penerbit,
            pustaka[idxMax].tahun)
    }

    // UrutBuku mengurutkan array pustaka berdasarkan rating secara menurun (descending)
    // menggunakan algoritma insertion sort
    func UrutBuku(pustaka *DaftarBuku, n int) {
        for i := 1; i < n; i++ {
            key := pustaka[i]
            j := i - 1
            for j >= 0 && pustaka[j].rating < key.rating {
                pustaka[j+1] = pustaka[j]
                j--
            }
            pustaka[j+1] = key
        }
    }

    // Cetak5Terbaru menampilkan 5 judul buku dengan rating tertinggi
    func Cetak5Terbaru(pustaka DaftarBuku, n int) {
        batas := 5
        if n < batas {
            batas = n
        }
        for i := 0; i < batas; i++ {

```

```

        fmt.Printf("%s ", pustaka[i].judul)
    }
    fmt.Println()
}

// CariBuku mencari dan menampilkan buku dengan rating
tertentu menggunakan binary search
func CariBuku(pustaka DaftarBuku, n int, r int) {
    // Binary search pada array yang sudah terurut
    descending berdasarkan rating
    kiri, kanan := 0, n-1
    ditemukan := false

    for kiri <= kanan {
        tengah := (kiri + kanan) / 2
        if pustaka[tengah].rating == r {
            // Tampilkan data buku yang ditemukan
            fmt.Printf("%s %s %s %d %d %d\n",
                pustaka[tengah].judul, pustaka[tengah].penulis,
                pustaka[tengah].penerbit,
                pustaka[tengah].tahun, pustaka[tengah].eksemplar,
                pustaka[tengah].rating)
            ditemukan = true
            break
        } else if pustaka[tengah].rating < r {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    if !ditemukan {
        fmt.Println("Tidak ada buku dengan rating
seperti itu")
    }
}

func main() {
    var n int
    DaftarkanBuku(&pustaka, &n)

    // Cetak buku terfavorit (rating tertinggi) sebelum
    diurutkan
    CetakTerfavorit(pustaka, n)

    // Urutkan buku berdasarkan rating (menurun)
    UrutBuku(&pustaka, n)

    // Cetak 5 judul buku dengan rating tertinggi
    Cetak5Terbaru(pustaka, n)
}

```

```

        // Baca rating yang dicari
        var ratingDicari int
        fmt.Scan(&ratingDicari)

        // Cari buku dengan rating tersebut
        CariBuku(pustaka, n, ratingDicari)
    }

```

Output :

```

PS D:\KULIAH\SEMESTER 2\Algoritma & Pemrograman 2\Praktikum\109082530026_Rista-Sania-Putri\Week
12 - Modul 14 SORTING\Unguided\Insertion Sort\unguided-no2> go run no2.go
6
B01 LaskarPelangi AndreaHirata Bentang 10 2005 95
B02 AtomicHabits JamesClear Gramedia 8 2018 90
B03 CleanCode RobertMartin Prentice 5 2008 98
B04 HarryPotter JKRowling Bloomsbury 12 1997 92
B05 Algoritma Daspro Informatika 7 2022 88
B06 PemrogramanGo Google OReilly 6 2020 85
CleanCode RobertMartin Prentice 2008
CleanCode LaskarPelangi HarryPotter AtomicHabits Algoritma
90
AtomicHabits JamesClear Gramedia 2018 8 90

```

Penjelasan :

Program ini mengelola data buku perpustakaan dengan 7 buku berjudul karya sastra Indonesia seperti "Bumi", "Negeri Para Bedebah", "Pulang", dan lainnya. Buku terfavorit adalah "Bumi" dengan rating 98. Setelah diurutkan, 5 buku dengan rating tertinggi adalah Bumi (98), Negeri Para Bedebah (97), Pulang (96), Tenggelamnya Kapal Van Der Wijck (94), dan Rindu (93). Pencarian rating 93 berhasil menemukan buku "Rindu" dengan detail lengkap. Program ini menggunakan insertion sort untuk pengurutan dan binary search untuk pencarian, sehingga efisien meskipun dengan data yang banyak.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa selection sort dan insertion sort memiliki pendekatan yang berbeda dalam mengurutkan data. Selection sort bekerja dengan mencari nilai ekstrim (terkecil/terbesar) lalu menukarnya ke posisi yang tepat, sehingga jumlah pertukaran minimal namun selalu $O(n^2)$. Insertion sort bekerja dengan menyisipkan elemen ke posisi yang benar pada bagian data yang sudah terurut, bersifat adaptif ($O(n)$ untuk data hampir terurut) dan stabil. Pada praktikum ini, selection sort digunakan untuk mengurutkan nomor rumah kerabat, sedangkan insertion sort digunakan untuk mengelola data perpustakaan (mengurutkan buku berdasarkan rating secara descending dengan variabel sementara bertipe struct). Dari praktikum ini dapat dipahami bahwa algoritma sorting sangat penting dalam pengolahan data terstruktur agar informasi dapat ditampilkan secara sistematis serta mempermudah proses pencarian data di tahap selanjutnya.